Dept. of Electronics and Electrical Communication Engineering

Indian Institute of Technology, Kharagpur

VLSI Design Lab

Experiment-3B

Design and Implementation of a Structural 4-bit MAC Unit

Chiradip Biswas 22EC37004

Arup Basak 22EC37021

1 Introduction

The objective of this experiment is to design and implement a 4-bit Multiply-Accumulate (MAC) unit using a structural Verilog modeling approach. A MAC unit computes the product of two numbers and adds that product to an accumulated sum ($Z \leftarrow Z + (A \times B)$), forming the core computational block of digital signal processors (DSPs) and machine learning accelerators.

The design is built hierarchically:

1. **Multiplier:** A 4×4 array multiplier built structurally using full adders.
2. **Adder:** A 12-bit structural adder (reused from previous experiments) to add the 8-bit product to the current accumulator value.
3. **Accumulator:** A 12-bit sequential register to store the running sum.

The design is verified via simulation and synthesized to analyze its resource utilization and critical path timing.

2 Verilog Implementation

2.1 Base Components: Full Adder and Multiplier

The multiplier utilizes an array structure (similar to a Braun multiplier) where partial products are generated using AND gates and summed using an array of Full Adders.

```
module full_adder(
    input a, b, cin,
    output sum, cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (a & cin);
endmodule
```

Listing 1: Structural Full Adder

```
module multiplier_4x4_struct (
    input [3:0] a,
    input [3:0] b,
    output [7:0] p
);
    wire [3:0] g0, g1, g2, g3; // Partial products
    wire [3:0] s1, s2;         // Intermediate sums
    wire [3:0] c1, c2, c3;     // Intermediate carries

    // Generate all Partial Products (AND Array)
    assign g0 = a & {4{b[0]}};
    assign g1 = a & {4{b[1]}};
    assign g2 = a & {4{b[2]}};
    assign g3 = a & {4{b[3]}};

    // --- Row 1 (Adding g0 and g1) ---
```

```
    assign p[0] = g0[0];
    full_adder fa10 (g0[1], g1[0], 1'b0,  p[1],  c1[0]);
    full_adder fa11 (g0[2], g1[1], c1[0], s1[0], c1[1]);
    full_adder fa12 (g0[3], g1[2], c1[1], s1[1], c1[2]);
    full_adder fa13 (1'b0,  g1[3], c1[2], s1[2], s1[3]); // Vertical
   carry

    // --- Row 2 (Adding s1 and g2) ---
    full_adder fa20 (s1[0], g2[0], 1'b0,  p[2],  c2[0]);
    full_adder fa21 (s1[1], g2[1], c2[0], s2[0], c2[1]);
    full_adder fa22 (s1[2], g2[2], c2[1], s2[1], c2[2]);
    full_adder fa23 (s1[3], g2[3], c2[2], s2[2], s2[3]);

    // --- Row 3 (Adding s2 and g3 - Final Product Bits) ---
    full_adder fa30 (s2[0], g3[0], 1'b0,  p[3],  c3[0]);
    full_adder fa31 (s2[1], g3[1], c3[0], p[4],  c3[1]);
    full_adder fa32 (s2[2], g3[2], c3[1], p[5],  c3[2]);
    full_adder fa33 (s2[3], g3[3], c3[2], p[6],  p[7]); // p[7] is
   final carry out
endmodule
```

Listing 2: 4x4 Structural Array Multiplier

2.2 Top-Level MAC Module

The top module integrates the multiplier, a 12-bit adder, and a synchronous active-high reset block for the accumulator register.

```
module mac_structural (
    input clk,
    input reset,
    input [3:0] a,
    input [3:0] b,
    output reg [11:0] accum
);
    wire [7:0] multiplier_out;
    wire [11:0] next_accum;
    wire cout_unused;

    // 1. Structural Multiplier (4x4)
    multiplier_4x4_struct mult_inst (
        .a(a), .b(b), .p(multiplier_out)
    );

    // 2. Structural 12-bit Adder
    // multiplier_out is zero-extended to 12 bits: {4'b0,
   multiplier_out}
    adder_12bit acc_adder (
        .a(accum),
        .b({4'b0, multiplier_out}),
```

```
        .cin(1'b0),
        .sum(next_accum),
        .cout(cout_unused)
    );

    // 3. Register Logic (Sequential Accumulator)
    always @(posedge clk or posedge reset) begin
        if (reset)
            accum <= 12'b0;
        else
            accum <= next_accum;
    end
endmodule
```

Listing 3: Top-Level MAC Unit

2.3 Testbench Implementation

The testbench provides a clock signal, initializes a reset, and runs various test cases to verify accumulation limits and reset behavior during active calculations.

```
module tb_mac_structural;
    reg clk;
    reg reset;
    reg [3:0] a, b;
    wire [11:0] accum;

    mac_structural uut (.clk(clk), .reset(reset), .a(a), .b(b), .accum(
    accum));

    // Clock generation (100MHz)
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; a = 0; b = 0;
        #100; reset = 0;

        // Test Case 1: Simple Multiplication (3 * 2 = 6) -> Accum = 6
        a = 4'd3; b = 4'd2; #100;

        // Test Case 2: Continuous Accumulation (4 * 5 = 20) -> Accum =
    26
        a = 4'd4; b = 4'd5; #100;

        // Test Case 3: Max Values (15 * 15 = 225) -> Accum = 251
        a = 4'd15; b = 4'd15; #100;

        // Test Case 4: Zero Multiplication (10 * 0 = 0) -> Accum = 251
        a = 4'd10; b = 4'd0; #10;

```

```
28          // Test Case 5: Reset during operation -> Accum = 0
29          reset = 1; #100; reset = 0;
30
31          #150; $finish;
32      end
33  endmodule
```

Listing 4: MAC Testbench

3 Simulation Results

The post-implementation timing simulation verifies the correct arithmetic sequence. As observed in the waveform, the accumulator successfully adds the products of incoming data continuously at every clock cycle. Notably, following $3 \times 2 = 6$, the input changes to $4 \times 5 = 20$, and the accumulator correctly transitions to 26. It reaches 251 upon adding $15 \times 15 = 225$, and perfectly clears to 0 when the reset signal is asserted.

Figure 1: Post-Implementation Simulation Waveform for MAC Unit

4 Synthesis Results

The structural MAC design was synthesized using Xilinx Vivado targeting the `xc7k70tfbg484-3` device.

4.1 Resource Utilization and Timing Data

Table 1 summarizes the resource consumption and the critical path setup timing delay extracted from the synthesis reports.

Table 1: Resource Utilization and Critical Path Timing for MAC Unit

Metric	Value
Slice LUTs	31
Slice Registers	12
Bonded IOBs	22
BUFGCTRL	1
Critical Path Setup Delay (ns)	**3.030**

5 Discussions

Arup Basak 22EC37021

The design and implementation of the structural MAC unit successfully demonstrates the integration of purely combinational elements (multiplier and adder) with sequential logic (register) to form a complex datapath.

- **Timing and Critical Path Analysis:** Based on the timing report, the critical path exhibits a total delay of **3.030 ns**. The path originates from the input pin `a[1]`, travels through the combinational logic of the 4×4 structural array multiplier, ripples through the 12-bit structural adder, and terminates at the `D` input of the 12th flip-flop (`accum_reg[11]`). This delay translates to a maximum theoretical operating frequency well over 300 MHz, largely due to the relatively shallow logic depth of the 4-bit array multiplier.

- **Resource Utilization Analysis:** The hardware allocation strictly aligns with theoretical expectations. Exactly **12 Slice Registers** were inferred, corresponding flawlessly to our 12-bit accumulator requirement. The combinational logic required **31 Slice LUTs**, which efficiently encapsulates both the array multiplier (which utilizes multiple full adders) and the 12-bit carry-lookahead/ripple adder logic. The **22 Bonded IOBs** perfectly account for the system inputs (4+4 for data, 1 clock, 1 reset) and the 12-bit output.

- **Architectural Scalability:** Implementing the multiplier structurally rather than behaviorally (e.g., `a * b`) provides deeper insight into gate-level delays. However, while this array structure is efficient for 4×4 inputs, scaling it to larger bit-widths (e.g., 16-bit or 32-bit MAC units) using ripple-carry logic would exponentially increase the critical path delay. In such scenarios, implementing advanced multiplier architectures like Wallace Trees or Booth multipliers, coupled with pipelined registers between the multiplier and adder stages, would be required to maintain high clock frequencies.

Chiradip Biswas 22EC37004

- In this experiment we implemented a 8 bit mac unit that takes previous result, and 2 current inputs. It multiplies the 2 current inputs and adds them to the previous result (accumulating).

- In the process, we made a 8bit adder using 2 4bit carry look ahead adders connected in ripple carry adder configuration.

- **Resource Utilization Summary:** The synthesized hardware (on FPGA: xc7k70tfbg484-3) perfectly matches the anticipated architectural requirements. The implementation consumes exactly 12 Slice Registers, which directly map to the 12-bit accumulator design. The combinational logic is efficiently packed into 31 Slice LUTs, fitting multiple full adders within the array multiplier as well as the 12-bit adder logic. Finally, the design utilizes 22 Bonded IOBs, accounting for the 10 input signals (2 4-bit data buses, clock, and reset) and 12-bit output bus.

- **Architectural Scalability:** Utilizing a structural approach rather than a behavioral is an efficient approach as timing complexity of structural was O(N) but behavioural is O(N*N). While this specific array topology is highly effective for a 4×4 configuration, scaling the ripple-carry logic to wider operands (such as 16-bit or 32-bit MAC units) would severely degrade timing performance (as the critical delay path becomes longer). To maintain high clock frequencies in larger designs, it would be necessary to adopt high-speed multiplication architectures, such as Booth or Wallace

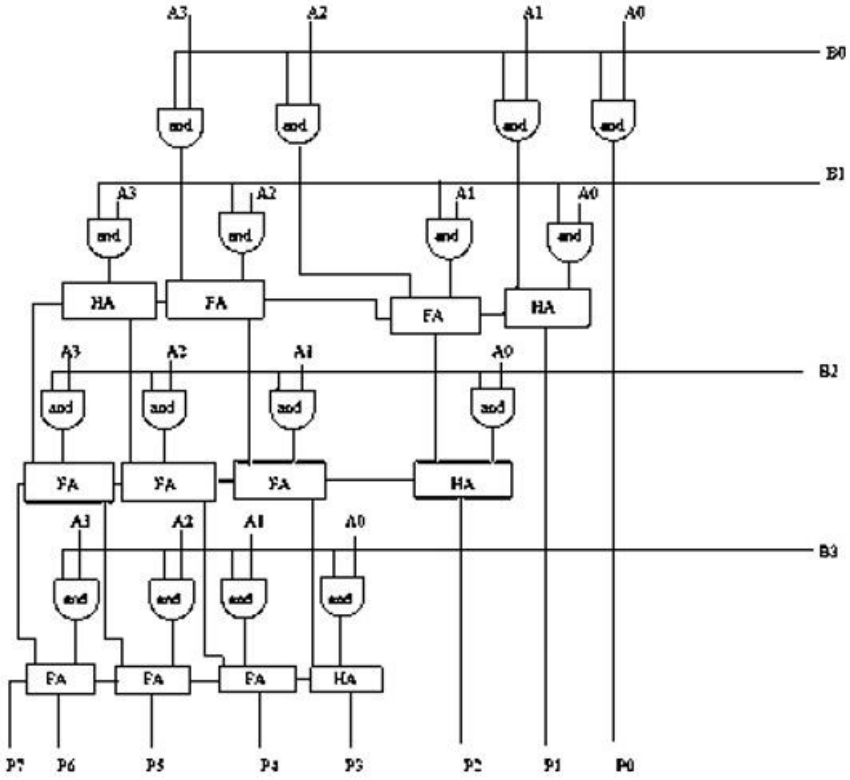

Figure 2: Braun array multiplier for 4bit x 4bit multiplication

Tree multipliers, alongside intermediate pipeline registers.

- **Critical Path and Timing Performance:** The timing summary report indicates a maximum critical path delay of 3.030 ns. This worst-case path begins at the input pin `a[1]`, propagates through the 4×4 structural multiplier's combinational logic, cascades across the 12-bit adder, and captures at the D-input of the final accumulator flip-flop (`accum_reg[11]`). Because the 4-bit multiplier possesses a relatively shallow logic depth, this delay supports a theoretical maximum operating frequency well above 300 MHz.